

movaddsubshift.S

```

1  @ =====
2  @ 範例：MOV（移動）、ADD（加法）、SUB（減法）、Shift（位移）指令的使用與說明
3  @ =====
4
5  .thumb_func      @ 必要宣告：因為 SDK 在呼叫此函式時會使用 BLX 指令切換至 Thumb 狀態
6  .global main     @ 宣告 main 為全域符號，提供程式起始位址給連結器（Linker）
7
8  main:    BL    stdio_init_all  @ 呼叫 SDK 函式，初始化 UART 或 USB 序列埠輸出
9
10 after:  MOV    R2, #3          @ 將立即值 3 存入暫存器 R2 (R2 = 3)
11        MOV    R1, R2          @ 將 R2 的值複製到 R1 (R1 變為 3)
12        MOVS   R1, R2          @ 帶有 S 的 MOV 指令，明確表示移動數值時同時更新 APSR 狀態旗標（如
Zero, Negative)
13        MOV    R9, R2          @ 將 R2 的值複製到高位暫存器 R9 (R9 = 3)
14
15 @ 在完全了解其運作機制之前，請勿隨意修改 SP（堆疊指標）或 PC（程式計數器）。
16 @ MOV    SP, R10      @ 範例：將 R10 的值寫入 SP (SP 即 R13)
17 @ MOV    PC, R11      @ 範例：將 R11 的值寫入 PC (PC 即 R15，會觸發跳躍)
18
19 ADD    R4, R2, #7        @ 將 R2 的值加上 7 存入 R4。此格式的立即值僅支援 3 位元（範圍 0-7）
20 @ 此時 R4 = 3 + 7 = 10 (0xA)
21
22 ADD    R4, R4, #255      @ 將 R4 的值加上 255 存入 R4。此格式支援 8 位元立即值（範圍 0-255）
23 @ 此時 R4 = 10 + 255 = 265 (0x109)
24
25 ADD    R4, #255          @ 簡寫形式，等同於 R4 = R4 + 255
26 @ 此時 R4 = 265 + 255 = 520 (0x208)
27
28 MOV    R7, #23           @ Thumb 指令集無法直接將立即值載入至高位暫存器 (R8-R15)
29 MOV    R11, R7           @ 因此先載入到低位暫存器 R7，再複製到高位暫存器 R11 (R11 = 23)
30
31 MOV    R7, #54           @ 將 54 載入 R7
32 MOV    R10, R7           @ 將 R7 的值複製到高位暫存器 R10 (R10 = 54)
33
34 ADD    R10, R10, R11     @ 將 R10 與 R11 相加並存回 R10。這是少數允許對高位暫存器進行操作的指令
35 @ 此時 R10 = 54 + 23 = 77 (0x4D)
36
37 ADD    SP, SP, #508      @ 調整堆疊指標 SP（將堆疊空間往上移 508 位元組，通常不建議單獨執行）
38 SUB    SP, SP, #508      @ 扣回 508 位元組，復原對堆疊指標 SP 的改動（維持堆疊平衡）
39
40 ADD    R4, SP, #1020     @ 將 SP 的值加上 1020 存入 R4。此處立即值支援 8 位元，但必須是 4 的倍數
(範圍 0-1020)
41 @ R4 的最終數值取決於當時 SP 的狀態，需透過除錯器（Debugger）來確認。
42 @ 例如作者執行時得到的結果為 0x200423fc。
43
44 @ =====
45 @ 接下來使用組合語言助憶符（Mnemonics）來進行位移運算
46 @ =====
47
48 MOV    R3, #8            @ R3 = 8，稍後用來進行 1 個位元組（8 bits）的位移或旋轉
49 MOV    R2, #0xFF         @ R2 = 255（十六進位 0xFF）
50 MOV    R1, #4            @ R1 = 4

```

```

51
52     LSL R1, R1, #2      @ 將 R1 的值向左位移 2 位元，並存入 R1 (相當於乘以 4)。此時 R1 = 4 * 4
    = 16
53     LSL R1, #2          @ 簡寫形式 (來源與目的地相同)：將 R1 再左移 2 位元。此時 R1 = 16 * 4 =
    64
54
55     LSR R1, R2, #8      @ 將 R2 的值向右邏輯位移 8 位元 (高位補 0)，結果存入 R1。0xFF 右移 8 位
    元後 R1 = 0
56     LSR R1, R3          @ 將 R1 的值向右邏輯位移 R3 (即 8) 個位元，結果存入 R1
57     ASR R1, #8          @ 將 R1 的值向右算術位移 8 位元 (保留正負號位元，若為負數高位補 1)
58     ROR R1, R3          @ 將 R1 的值向右循環旋轉 R3 (即 8) 個位元
59
60 @ =====
61 @ 透過分段加法，手動將 32 位元立即值 0x12345678 載入至 R3
62 @ =====
63
64     MOV R3, #0x12        @ 載入最左側 (最高位) 的第 1 個位元組 0x12 到 R3
65     LSL R3, #24          @ 將 R3 左移 24 位元，移動到最左側位置 (R3 = 0x12000000)
66
67     MOV R1, #0x34        @ 載入第 2 個位元組 0x34 到 R1
68     LSL R1, #16          @ 將 R1 左移 16 位元，移動到對應位置 (R1 = 0x00340000)
69     ADD R3, R1           @ 將其加進 R3 中 (R3 = 0x12340000)
70
71     MOV R1, #0x56        @ 載入第 3 個位元組 0x56 到 R1
72     LSL R1, #8           @ 將 R1 左移 8 位元，移動到對應位置 (R1 = 0x00005600)
73     ADD R3, R1           @ 將其加進 R3 中 (R3 = 0x12345600)
74
75     MOV R1, #0x78        @ 載入最後 1 個位元組 0x78 到 R1 (最右側，不需位移)
76     ADD R3, R1           @ 將其加進 R3 中。此時 R3 成功拼湊為完整的 0x12345678
77
78 @ =====
79 @ 為接下來的 64 位元加法初始化其他暫存器
80 @ =====
81
82     MOV R2, #0x12        @ R2 = 0x00000012 (作為加數 1 的高 32 位元)
83     MOV R4, #0x54        @ R4 = 0x00000054 (作為加數 2 的高 32 位元)
84
85     MOV R5, #0xf0        @ R5 = 0x000000f0
86     LSL R5, #24          @ 將 R5 左移 24 位元，使其變為 0xf0000000 (作為加數 2 的低 32 位元)
87
88 @ =====
89 @ 64 位元加法實作 (刻意設計會產生進位 Carry)
90 @ 運算邏輯：
91 @      R2 R3    0x00000012 0x12345678
92 @      + R4 R5    0x00000054 0xf0000000
93 @      -----
94 @      R0 R1    0x00000067 0x02345678
95 @
96 @ 說明：低 32 位元相加 (0x12345678 + 0xf0000000) 會超過 32 位元上限 (0xffffffff)，
97 @ 進而產生溢位 (進位至第 33 位元)，並使 APSR 的 C (Carry) 旗標被設為 1。
98 @ =====
99
100    ADD R1, R3, R5        @ 計算低 32 位元相加：R1 = R3 + R5 (R1 變為 0x02345678，並產生進位旗
    標 C=1)

```

```

101     ADC R2, R4          @ 計算高 32 位元帶進位加法：R2 = R2 + R4 + Carry (即 0x12 + 0x54 + 1
    = 0x67)
102     MOV R0, R2          @ 將高位運算結果複製到 R0，此時 R0 = 0x67
103
104 @ =====
105 @ 備份計算結果，因為呼叫 printf 函式時會覆寫 R0 - R3 暫存器
106 @ =====
107
108     MOV R6, R0           @ 將高 32 位元結果備份到 R6 (R6 = 0x67)
109     MOV R7, R1           @ 將低 32 位元結果備份到 R7 (R7 = 0x02345678)
110
111 @ =====
112 @ 無限循環列印加法結果
113 @ =====
114
115 loop: MOV R1, R6         @ 將 R6 (高 32 位元結果) 放入 R1，作為 printf 的第二個參數
116     MOV R2, R7           @ 將 R7 (低 32 位元結果) 放入 R2，作為 printf 的第三個參數
117     LDR R0, =sumstr      @ 將格式化字串 sumstr 的記憶體位址載入到 R0，作為 printf 的第一個參數
118     BL printf            @ 呼叫 printf 函式輸出結果到終端機
119     B loop               @ 重新跳回 loop 標記處，防止因 UART 監控未即時啟動而漏看輸出
120
121 .data
122     .align 4             @ 記憶體 4 位元組對齊 (提升 CPU 讀取效率並避免對齊錯誤)
123 sumstr: .asciz "The sum is %x %x\n" @ 定義以空字元 \0 結尾的 C 語言格式化字串

```

